Computer Graphics

< TheAviator >

담당 교수 : 김동준 교수님

학번 : 2021204010

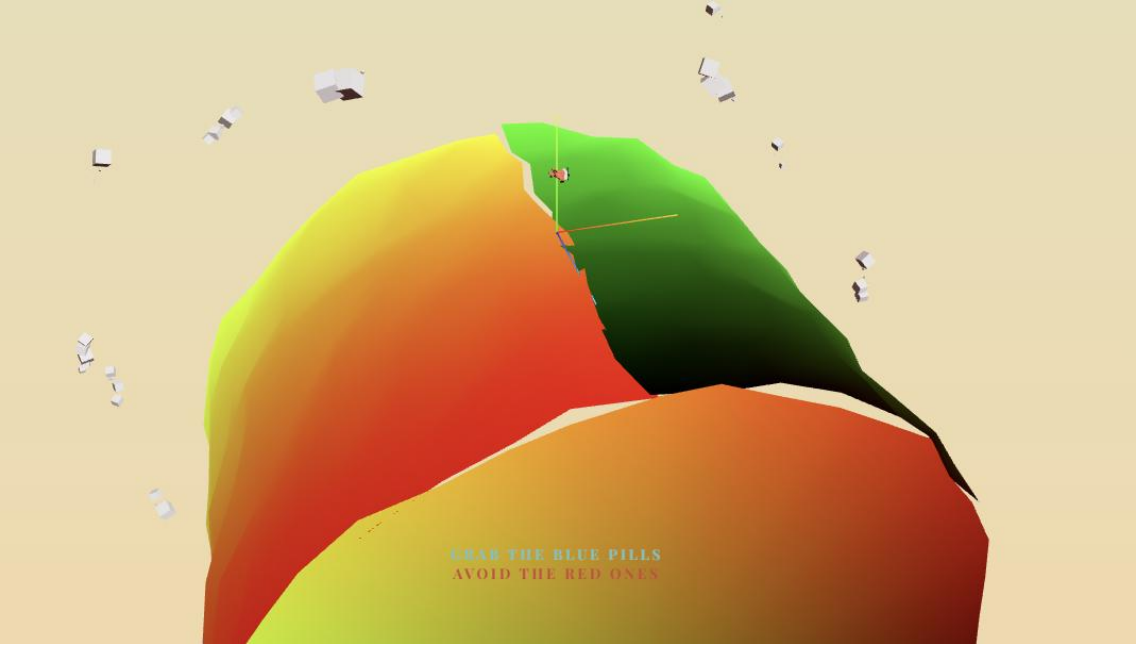
학과 : 정보융합학부

이름 : 박현준

목차

- 1. Sea 의 geometry 를 vertex shader 에서 변형
- 2. WASD key 기반 비행기 조정 구현
- 3. 비행기 뒤쪽 3인칭 게임용 카메라
- 4. 결론

1-1. Vertex Shader 적용 전 Sea Geometry 문제점



[문제 해결 전 Sea geometry]

1. UV 기반 랜덤 → 시각적 불연속

```
float ang = random(uv) * 3.14f * 2.f;
```

- random(uv)로 인해 이웃 정점마다 달라짐

2. mergeVertices() 전 UV 삭제

```
delete geom.attributes.uv;
delete geom.attributes.normal;
// 정점 병합
geom = mergeVertices(geom);
```

- UV 정보가 사라져 Shader에서 uv 값이 (0,0)으로 고정됨

1-2. Vertex Shader 기반 Sea Geometry 변형 – 해결 방법

[해결 방법] GPU 기반 Shader 처리로 전환

1. **파형 계산 위치 변경**
   - CPU → GPU(Vertex Shader)로 파형 계산 책임 이동
   - Sea 파도 애니메이션 전체를 Vertex Shader에서 처리

2. **Seed 변경: uv → position.xz**
   - uv의 단점(끊김, 삭제 위험)을 피하기 위한 위치 기반 Seed 사용
   - position.xz * 0.1로 좌표를 scale하여 random 분포 조절
   - 동일한 정점 위치에는 항상 같은 파형이 적용되어 연속성 유지

1-2. Vertex Shader 기반 Sea Geometry 변형 – 해결 방법

[주요 Uniform 설명]

```
let delta_variation = 0;
let my_uniforms = {
  'my_color': { value: new THREE.Vector3(1, 0, 0) },
  'wavesMinAmp': { value: 5.0 },
  'wavesMaxAmp': { value: 20.0 },
  'delta': { value: delta_variation }
};
```

- **delta** : 정점 위치를 시간에 따라 회전시켜 파형 애니메이션을 만들기 위한 시간 누적값
- **wavesMinAmp, wavesMaxAmp** : 정점별 진폭 범위 제어

1-3. Vertex Shader 기반 파형 애니메이션 처리

1. position 기반 랜덤 seed 생성

```
vec2 seed = position.xz * 0.1;
```

- uv를 seed 로 사용 → uv 끊김 현상 발생
- position.xz 사용 → vertex 좌표 기반이라 끊김 현상 발생 x
- position*0.1 → 너무 큰 값을 줄일 수 있는 scale factor

2. 회전 각도와 진폭 계산

```
float baseAng = random(seed) * 6.28318530718; // 2*PI
float amp     = wavesMinAmp + random(seed + 37.1) * (wavesMaxAmp - wavesMinAmp);
float ang = baseAng + delta;
```

- baseAng: 정점의 초기 회전 각도(랜덤)
- amp: 정점마다 다른 진폭 (0~20 사이)
- random(seed + 37.1): 다른 속성(진폭)을 위해 baseAng와 구분되는 독립적인 random값을 생성
- delta: 현재 시간값 → 시간에 따라 ang이 점점 증가 → **회전 애니메이션 구현**

1-3. Vertex Shader 기반 파형 애니메이션 처리

3. 정점 위치 이동 및 출력

```
vec3 pos_new = position;
pos_new.x += cos(ang) * amp;
pos_new.y += sin(ang) * amp;

gl_Position = projectionMatrix * modelViewMatrix * vec4(pos_new, 1.0);
```

- pos_new.x/y : 회전 각도에 따라 위치를 조절 (원 궤도 따라 흔들림)
- Z축은 그대로 → 수평 파도 느낌 유지
- 최종적으로 gl_Position에 넘겨 렌더링

4. function Sea()

```
function Sea() {
  let geom = new THREE.CylinderGeometry(game.seaRadius, game.seaRadius, game.seaLength, 40,
  10);
  geom.rotateX(-Math.PI / 2);

  this.mesh = new THREE.Mesh(geom, my_material);
  this.mesh.name = "waves";
  this.mesh.receiveShadow = true;
}
```

- uv 속성 제거 및 병합, 노멀 재계산 코드 제거
- CPU 파도 데이터 초기화 코드 제거

1-3. Vertex Shader 기반 파형 애니메이션 처리

5. 시간 누적과 uniform 갱신

```
delta_variation += deltaTime * 0.001;

my_uniforms.delta.value = delta_variation;
```

[Function loop() 내에 존재]

- **deltaTime** : 프레임 간 경과 시간(ms 단위)
- **delta_variation** : 누적 시간 (초 단위로 변환하여 누적)
- **my_uniform.delta.value** : Shader uniform 값에 반영 → Vertex Shader에서 회전값(ang)에 시간 적용됨

[설명 요약]

- Vertex Shader 내 파형 각도(ang) = baseAng + delta
- **delta가 시간에 따라 계속 증가하면서** → 정점마다의 파형이 회전하며 **애니메이션 효과** 발생

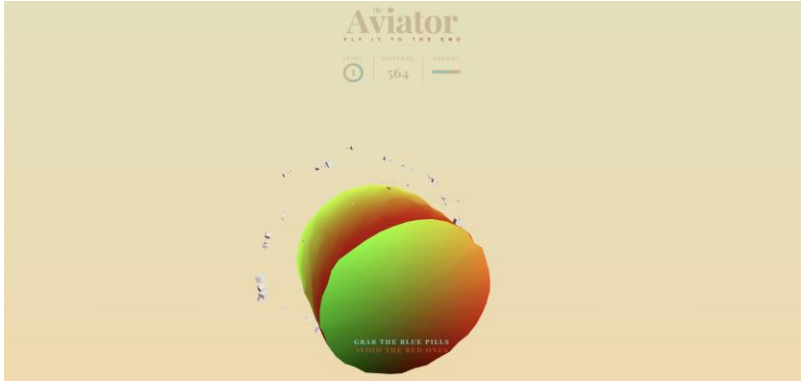
→ GPU 기반의 파형 애니메이션 구현

1-3. Vertex Shader 기반 파형 애니메이션 처리

5. 전체 코드 구조

```
let my_vtxShader = `
    uniform float wavesMinAmp;
    uniform float wavesMaxAmp;
    uniform float delta;
    varying vec2 vUv;

    // position.xz 로 seed
    float random(vec2 st) {
        return fract(sin(dot(st, vec2(12.9898,78.233))) * 43758.5453123);
    }

    void main() {
      // 1) uv는 그대로 넘겨서 fragShader 에서 색 계산에 사용
      vUv = uv;

      // 2) position 기반 seed (실린더 끊김 없음)
      vec2 seed = position.xz * 0.1;
      float baseAng = random(seed) * 6.28318530718; // 2*PI
      float amp     = wavesMinAmp + random(seed + 37.1) * (wavesMaxAmp - wavesMinAmp);

      // 3) 시간(delta)을 더해서 애니메이션 회전
      float ang = baseAng + delta;

      // 4) 변형 적용
      vec3 pos_new = position;
      pos_new.x += cos(ang) * amp;
      pos_new.y += sin(ang) * amp;

      // 5) standard MVP
      gl_Position = projectionMatrix * modelViewMatrix * vec4(pos_new, 1.0);
    }
`;
```

6. 결과 사진



결과 요약: 파형이 끊김 없이 연속적으로 애니메이션됨, GPU 기반으로 성능 향상

2. WASD key 기반 비행기 조정

[구현 과정]

- 사용자의 WASD 키 입력에 따라 비행기를 world 기준 **지정된 거리(MOVE_DIST = 300)만큼 이동**
- 각 **방향에 따라 회전**도 함께 적용 (TILT_ANGLE = π / 1.5)
- Quaternion 기반 **SLERP(Spherical Linear Interpolation)**를 통해 부드러운 회전 적용
- 이동 중에는 **반대 방향 입력 시 즉시 복귀**, **직교 방향 입력은 큐에 저장하여 연속 조작 지원**

2. WASD key 기반 비행기 조정

[핵심 변수 및 초기 설정]

```
const MOVE_DIST = 300;
const TILT_ANGLE = Math.PI / 1.5;

let isMoving = false;      // WASD 이동 중
let moveDir = null;       // 'w'|'a'|'s'|'d'
let originPos = new THREE.Vector3();   // 출발점(복귀용)
let startPos = new THREE.Vector3();   // 보간 시작
let targetPos = new THREE.Vector3();   // 보간 목표
let startQuat = new THREE.Quaternion();
let targetQuat = new THREE.Quaternion();
const endQuat = new THREE.Quaternion();  // 수평(Identity)
let progress = 0;           // 0 → 1 구간

const opposite = { w: 's', s: 'w', a: 'd', d: 'a' };
```

- **MOVE_DIST** : 이동할 거리 (300)
- **TILT_ANGLE** : 회전 기울기 각도
- **originPos** : 현재 위치 저장(복귀시 사용)
- **startPos, targetPos** : 위치 보간용 시작/ 도착 지점
- **endQuat** : 회전 복귀 시 목표 회전 (기본값은 수평)
- **progress** : 이동의 진행률(시간에 따라 0에서 1로 증가)

2-1. 방향 및 회전 계산 함수 정의

```
function dirVector(key) {
  switch (key) {
    case 'w': return new THREE.Vector3(0, 1, 0);   // +Y
    case 's': return new THREE.Vector3(0, -1, 0);   // -Y
    case 'a': return new THREE.Vector3(0, 0, -1);   // -Z
    case 'd': return new THREE.Vector3(0, 0, 1);   // +Z
  }
}
function tiltAxis(key) {
  // W/S 는 Z-축 , A/D 는 X-축
  return (key === 'w' || key === 's') ? new THREE.Vector3(0, 0, 1)
    : new THREE.Vector3(1, 0, 0);
}
function newTiltQuat(key) {
  const axis = tiltAxis(key);
  const angle = (key === 'w' || key === 'd') ? TILT_ANGLE : -TILT_ANGLE;
  return new THREE.Quaternion().setFromAxisAngle(axis, angle);
}

function makeReturnQuat(currentQuat, key) {
  const axis = tiltAxis(key);        You, 4 minutes ago • Uncommitted changes
  const angle = (key === 'w' || key === 'd') ? -TILT_ANGLE : TILT_ANGLE;
  const returnTilt = new THREE.Quaternion().setFromAxisAngle(axis, angle);
  return currentQuat.clone().multiply(returnTilt);
}
```

- **derVector(key)** : 입력된 키에 따라 **이동 방향 벡터**를 반환

- **tiltAxis(key)** : 비행기가 움직일 때 **어떤 축을 기준으로 회전(기울이기)** 할지를 결정
  - w/s 입력 → 위아래 이동 → Z축 기준 회전
  - a/d 입력 → 좌우 이동 → X축 기준 회전

- **newTiltQuat(key)** : 현재 쿼터니언 회전에 "기울임 회전"을 추가하기 위한 쿼터니언 생성

- **makeReturnQuat(currentQuat, key)** : **반대 방향 키를 눌러서 원위치(수평 상태)로 복귀할 때** 필요한 쿼터니언을 생성

2-2. 이동 제어 흐름 - handleMoveKey(key)

```javascript
let nextQueuedKey = null;

function handleMoveKey(key) {
  key = key.toLowerCase();
  if (!'wasd'.includes(key)) return;

  if (isMoving) {
    // 반대 방향이면 즉시 복귀
    if (key === opposite[moveDir]) {
      moveDir = null;
      startPos.copy(airplane.mesh.position);
      startQuat.copy(airplane.mesh.quaternion);
      targetPos.copy(originPos);

      // 현재 회전 상태에서 "기울어짐 제거 회전"을 적용해서 최종 회전 만들기
      const inverseTilt = newTiltQuat(key).invert(); // 반대 방향 기울임
      targetQuat.copy(startQuat).multiply(inverseTilt);
      progress = 0;
    } else {
      // 동일/직교 방향은 예약만 (중복 저장은 방지)
      if (!nextQueuedKey) {
        nextQueuedKey = key;
      }
    }
    return;
  }
  // 새 이동 시작
  startMove(key);
}
```

```javascript
document.addEventListener('keydown', (e) => {
  handleMoveKey(e.key);    // ← 방금 만든 함수 호출
});
```

사용자의 WASD 키 입력에 따라 비행기 이동을 제어하며, 현재 이동 상태에 따라 다음과 같이 분기 처리:

1. 반대 방향 입력 시 → 즉시 복귀 처리

- 현재 이동 중인 방향과 반대되는 키가 입력되면, 현재 위치·회전을 시작점으로 하고, 원래 위치(originPos)로 돌아가도록 목표 위치와 회전을 설정

2. 동일 또는 직교 방향 입력 시 → 큐에 저장

- 이동 중 동일, 직교 방향 키가 눌리면 즉시 처리 X nextQueueKey에 저장, 현재 이동이 완료된 후 자연스럽게 이어지도록 예약

3. 이동 중이 아닐 경우 → 이동 즉시 시작

- 현재 이동 중이 아니라면 **startMove(key)를 호출해 바로 이동을 시작**

이동 중 키 입력을 유연하게 처리하여 **반응성**과 **연속성**을 확보

2-2. 이동 제어 흐름 - startMove(key)

```
function startMove(key) {
  isMoving = true;
  moveDir = key;
  originPos.copy(airplane.mesh.position);   // 기준점 갱신
  startPos.copy(originPos);
  startQuat.copy(airplane.mesh.quaternion);

  targetPos.copy(originPos).addScaledVector(dirVector(key), MOVE_DIST);
  targetQuat.copy(startQuat).multiply(newTiltQuat(key));
  progress = 0;
}
```

새로운 WASD 키 입력을 받아 비행기 이동을 시작할 때 **초기 상태를 세팅**

1. 이동 상태 플러그 설정

- **isMoving = true**로 현재 이동 중임을 명시, 현재 방향 키 (moveDir)도 저장

2. 현재 상태 백업 (출발점 저장)

- **originPos** : 현재 비행기 위치 저장
- **startPos** : 보간 시작점으로 사용
- **startQuat** : 현재 회전 상태 저장

3. 목표 위치, 회전 계산

- **targetPos** : 방향벡터(dirVector(key))로 MOVE_DIST만큼 이동한 목표 위치 계산
- **targetQuat** : 해당 방향의 기울기를 적용한 쿼터니언 생성

4. 보간 진행률 초기화

- **Progress = 0**으로 설정하여 새로운 이동의 보간이 0부터 시작

2-2. 이동 제어 흐름 – updateWASD(dt)

```javascript
function updateWASD(dt) {
  if (!isMoving) return;

  progress = Math.min(progress + dt * 0.0015, 1);

  //위치 보간
  airplane.mesh.position.lerpVectors(startPos, targetPos, progress);
  purePos.copy(airplane.mesh.position);          // ← 순수 위치 기록

  //회전 보간
  if (progress < 0.3) {
    let q1 = startQuat;
    let q2 = targetQuat;
    let q2s = q2.clone();
    if (q1.dot(q2) < 0) {
      q2s.set(-q2.x, -q2.y, -q2.z, -q2.w);  // negate와 동일한 결과
    }

    airplane.mesh.quaternion.slerpQuaternions(q1, q2s, progress / 0.3);

  } else if (progress < 0.7) {
    airplane.mesh.quaternion.copy(targetQuat);

  } else {
    let q1 = targetQuat;
    let q2 = endQuat;
    let q2s = q2.clone(); // ← 먼저 clone을 따로 실행하고
    if (q1.dot(q2) < 0) {
      q2s.set(-q2.x, -q2.y, -q2.z, -q2.w);  // negate와 동일한 결과
    }

      airplane.mesh.quaternion.slerpQuaternions(q1, q2s, (progress - 0.7) / 0.3);
    }
    // 완료
    if (progress >= 1) {
      airplane.mesh.position.copy(targetPos);
      airplane.mesh.quaternion.copy(endQuat);
      isMoving = false;
      moveDir = null;

      // 도착 후 예약된 이동이 있다면 즉시 실행
      if (nextQueuedKey) {
        const keyToMove = nextQueuedKey;
        nextQueuedKey = null;
        startMove(keyToMove);
      }
    }      You, 43 minutes ago • Uncommitted changes
  }
```

시간 흐름(dt) 에 따라 비행기의 위치와 회전을 부드럽게 보간하여, 실제 비행기를 자연스럽게 움직이게 만드는 함수

1. 진행률(progress) 갱신

- Progress는 이동의 전체 진행률(0~1)을 나타냄, 점진적 증가

2. 위치 보간 (startPos → targetPos)

- 현재 위치를 시작점과 목표점 사이에서 선형 보간(Lerp)으로 계산하여 자연스럽게 이동
- **purePos**는 마우스 흔들림 없이 WASD 입력만 반영한 '기준 위치'로 저장됨

2-2. 이동 제어 흐름 – updateWASD(dt)

```javascript
function updateWASD(dt) {
  if (!isMoving) return;

  progress = Math.min(progress + dt * 0.0015, 1);

  //위치 보간
  airplane.mesh.position.lerpVectors(startPos, targetPos, progress);
  purePos.copy(airplane.mesh.position);          // ← 순수 위치 기록

  //회전 보간
  if (progress < 0.3) {
    let q1 = startQuat;
    let q2 = targetQuat;
    let q2s = q2.clone();
    if (q1.dot(q2) < 0) {
      q2s.set(-q2.x, -q2.y, -q2.z, -q2.w);  // negate와 동일한 결과
    }

    airplane.mesh.quaternion.slerpQuaternions(q1, q2s, progress / 0.3);

  } else if (progress < 0.7) {
    airplane.mesh.quaternion.copy(targetQuat);

  } else {
    let q1 = targetQuat;
    let q2 = endQuat;
    let q2s = q2.clone(); // ← 먼저 clone을 따로 실행하고
    if (q1.dot(q2) < 0) {
      q2s.set(-q2.x, -q2.y, -q2.z, -q2.w);  // negate와 동일한 결과
    }

      airplane.mesh.quaternion.slerpQuaternions(q1, q2s, (progress - 0.7) / 0.3);
    }
    // 완료
    if (progress >= 1) {
      airplane.mesh.position.copy(targetPos);
      airplane.mesh.quaternion.copy(endQuat);
      isMoving = false;
      moveDir = null;

      // 도착 후 예약된 이동이 있다면 즉시 실행
      if (nextQueuedKey) {
        const keyToMove = nextQueuedKey;
        nextQueuedKey = null;
        startMove(keyToMove);
      }
    }      You, 43 minutes ago • Uncommitted changes
  }
```

3. 회전 보간(Quaternion SLERP)

3.1 초기 기울기 구간 (0 ≤ progress < 0.3)

- 출발 회전에서 목표 회전까지 점진적으로 기울이기
- dot<0 보정으로 최단 거리 회전 경로 확보

3.2 기울기 유지 구간 (0.3 ≤ progress < 0.7)

- 목표 회전을 유지하여 비행기가 기울어진 상태를 유지

3.3 수평 복귀 구간 (0.7 ≤ progress ≤ 1.0)

- 목표 회전에서 수평 자세(endQuat)로 부드럽게 복귀

4. 이동 완료 처리 및 상태 초기화

- 위치와 회전을 정확히 목표 상태로 고정
- 이동 상태를 초기화하여 다음 입력을 받도록 준비

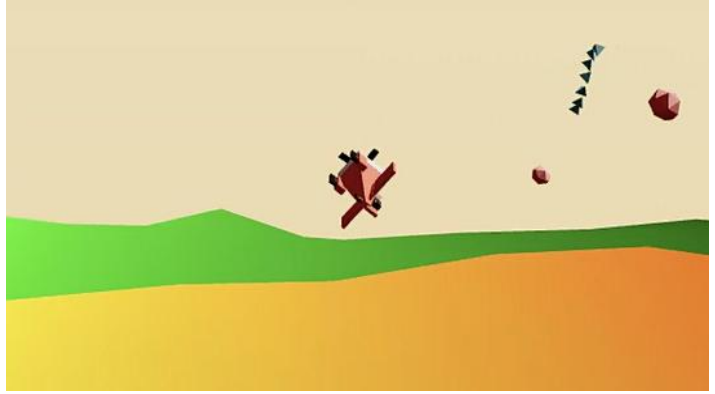
5. 다음 입력 예약 처리

- 이전에 큐에 저장해둔 방향 입력 있다면 다음 이동을 실행
- 연속된 WASD 조작을 자연스럽게 연결하여 부드러운 사용자 경험 제공

2-3 WASD key 기반 비행기 조정 **결과 사진**

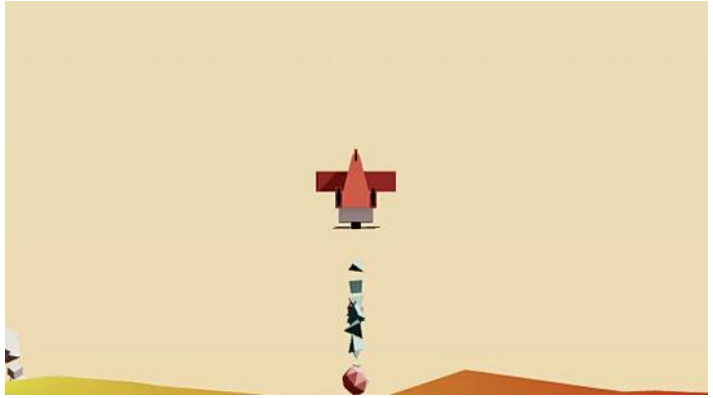
D 입력

W 입력



A → D 입력 (복귀)

A 입력



S 입력

D → A 입력 (복귀)

W/S, S/W도 동일하게 복귀 가능

3. 비행기 뒤쪽 3인칭 게임용 카메라

[카메라 구성 및 핵심 개념]

- 카메라를 비행기의 자식으로 두는 방식 대신, 별도의 **보조 객체(cameraAnchor)**를 생성하여 카메라를 그 자식으로 설정함
- 카메라 자체는 **회전에 영향을 받지 않고**, 위치와 시점을 독립적으로 제어 가능
- 비행기와 일정한 거리 및 높이를 유지하면서, 항상 앞을 바라보는 3인칭 시점을 구현

3. 비행기 뒤쪽 3인칭 게임용 카메라

[카메라 초기 구조]

```
let cameraAnchor = null;  // 비행기 뒤에 따라붙을 보조 객체
```

```
camera = new THREE.PerspectiveCamera(
  fieldOfView,
  aspectRatio,
  nearPlane,
  farPlane
);
```

```
cameraAnchor = new THREE.Object3D();
scene.add(cameraAnchor);      // 월드에 보조 객체 추가
cameraAnchor.add(camera);

camera.up.set(0, 1, 0);       // 항상 y축 위쪽 유지
```

1. cameraAnchor

- **비행기의 위치를 기준으로 카메라를 뒤쪽에 배치**하기 위한 중간 매개체

2. 카메라 생성

3. 보조 객체(cameraAnchor) 생성 및 연결

- 월드에 cameraAnchor 추가 후, cameraAnchor의 자식으로 카메라 연결

4. up vector 고정

- 카메라의 상단 방향을 항상 y축으로 유지하여 시점의 안정성 보장
- 회전 중에도 상하가 뒤집히지 않도록 고정

3. 비행기 뒤쪽 3인칭 게임용 카메라

```
function updateCamera() {
  const P = airplane.mesh.position;

  const DISTANCE_BACK = 400;   // 뒤로 떨어질 거리
  const HEIGHT = 50;    // 위로 띄울 높이
  cameraAnchor.position.set(
    P.x - DISTANCE_BACK,
    P.y + HEIGHT,
    P.z
  );

  const LOOK_AHEAD = 100;
  camera.lookAt(
    new THREE.Vector3(
      P.x + LOOK_AHEAD,
      cameraAnchor.position.y,
      P.z
    )
  );
}
```

비행기를 따라가는 3인칭 시점 카메라 제어 함수

1. 비행기 현재 위치 참조

- 비행기의 **현재 위치**를 기준으로 카메라 배치 및 시선 방향 계산 시작

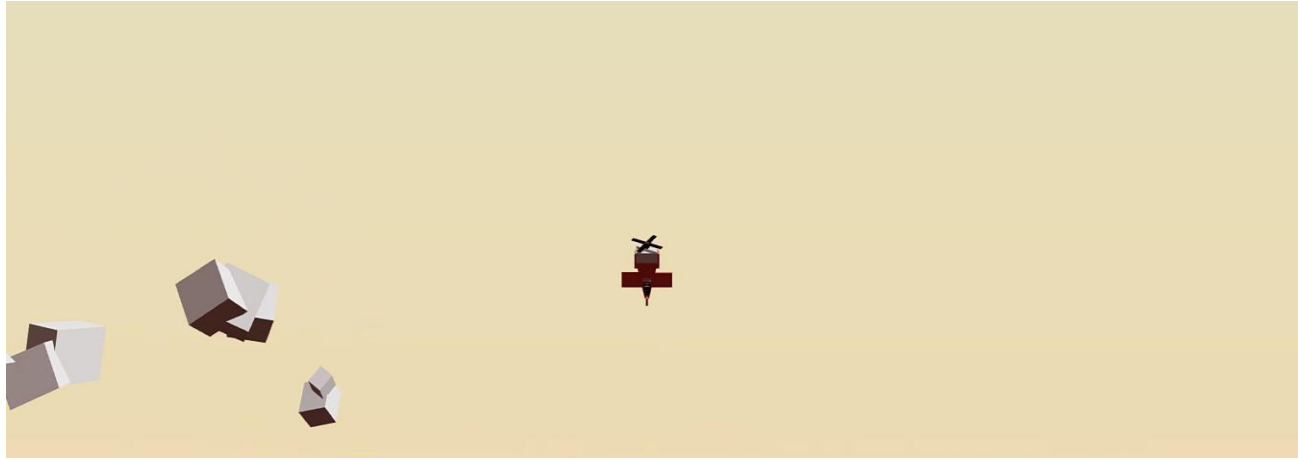
2. cameraAnchor 위치 지정

- 비행기보다 뒤쪽(DISTANCE_BACK), 위쪽(HEIGHT)에 카메라를 배치

3. 카메라 시선 지정

- 카메라는 항상 비행기 앞쪽(LOOK_AHEAD)을 **수평 방향으로 응시**
- Y좌표는 cameraAnchor의 높이에 맞춰, **기울기 없이 수평한 시선**을 응시

3. 비행기 뒤쪽 3인칭 게임용 카메라 결과 사진



4. 결론

1. vertex shader를 이용한 애니메이션, hole 처리

- uv 좌표계 유지
- deltaTime을 누적해 파도 애니메이션 구현
- GPU에서 vertex 위치를 지속적으로 변형 → **자연스러운 효과**

2. 직관적 이동 + 부드러운 회전

- 키 입력(WASD)에 따라 **정해진 거리 + 기울기** 이동
- **쿼터니안**기반 SLERP로 **자연스러운 회전 전환**

3. 반응성 높은 입력 처리

- **반대 방향 키** 입력 시 → 즉시 수평 복귀
- **직교 방향 키**는 큐에 저장 → 이동 종료 후 자동 실행
- 연속 조작에도 부드럽게 연결

4. 3인칭 카메라 구조 설계

- cameraAnchor를 이용해 **비행기 뒤쪽에 고정된 카메라** 구현
- Up vector로 **시점 안정성 확보**